

Ngôn ngữ PHP

NGUYỄN THỊ THÙY LIÊN

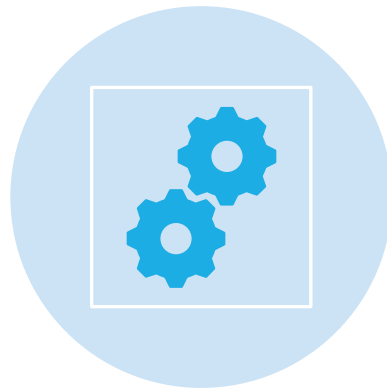
KHOA CNTT-ĐH PHENIKAA

LIEN.NGUYENTHITHUY@PHENIKAA-UNI.EDU.VN

Nội dung



NGÔN NGỮ PHÁT
TRIỂN WEB PHP



PHP FRAMEWORK



BẢO MẬT WEB VÀ
SEO

Giới thiệu ngôn ngữ lập trình PHP

Giới thiệu



- PHP: **P**HP **H**ypertext **P**reprocessor. Tên gốc là **P**ersonal **H**ome **P**age (Rasmus Lerdorf - 1994)
- Là ngôn ngữ script trên server để xây dựng các trang Web động.
- Bộ biên dịch PHP là phần mềm mã nguồn mở, có thể chạy cùng với các web server Apache, IIS,...
- Thường kết hợp với hệ quản trị CSDL MySQL

Nhúng PHP vào HTML

- Có thể nhúng mã PHP vào mọi vị trí trong trang HTML.
- Chèn mã PHP vào file HTML với các dạng:

<?php

//Đoạn lệnh PHP ở đây

?>

<?

//Đoạn lệnh PHP ở đây

?>

- Kết quả do đoạn lệnh PHP đưa ra được đưa vào vị trí mà đoạn lệnh PHP đang chiếm chỗ.

Đặc điểm PHP

- Có khả năng hướng đối tượng
- Thông dịch
- Không phân biệt chữ hoa/chữ thường với các từ khóa (`if`, `else`, `while`, `echo` ...), tên lớp, hàm
- Phân biệt chữ hoa/chữ thường với các biến
- Mỗi lệnh kết thúc bởi chấm phẩy (;)
- PHP là một ngôn ngữ kịch bản ràng buộc lỏng:
 - Không cần khai báo trước, việc khai báo sẽ được tự động thực hiện khi sử dụng.
 - Không cần định kiểu. Kiểu giá trị sẽ được xác định phù hợp với dữ liệu đầu vào.

Chú thích trong PHP

- // Dòng chú thích
- # Dòng chú thích
- /*

Đoạn chú thích trên nhiều dòng

```
*/  
    /*  
    * Author:      Joel Murach  
    * Purpose:    This program calculates the discount for  
    *              a price that's entered by the user  
    *              *****/  
    //lấy dữ liệu từ form  
    $list_price = $_GET['list_price'];
```

Khai báo và gán giá trị cho biến

- Khai báo biến:
 - Cú pháp: **\$tên_biến**
 - VD: \$a, \$b,...
- Quy tắc đặt tên biến:
 - Gồm chữ cái, chữ số, dấu gạch nối (_)
 - Phải bắt đầu bằng chữ cái hoặc dấu gạch nối.
 - Phân biệt chữ hoa, chữ thường
 - Biến không cần khai báo (được tự động khai báo vào lần gán giá trị đầu tiên).
 - Việc sử dụng biến chưa khởi tạo sẽ gây lỗi.

Khai báo và gán giá trị cho biến

- Gán giá trị cho biến: gán trực tiếp

\$ten_bien = <giá_trị>;

- Ví dụ:

\$price = 1000;

- Trong PHP có một số biến được định nghĩa sẵn phải sử dụng đúng mục đích thiết kế của nó.
- Biến có tên **\$this** không thực hiện phép toán gán được.

Khai báo và gán giá trị cho biến

- Biến động (biến biến)
 - Cho phép sử dụng giá trị của biến làm tên biến khác.

```
$a = "hello";
```

```
$$a = "world"; // $hello = "world"
```

```
<?php
```

```
    $_bien      = 'abc';
```

```
    $$_bien     = 'xyz';
```

```
    $$$_bien    = 'abc';
```

```
    $$$$_bien   = 'klm';
```

```
    echo $abc;
```

```
?>
```

Khai báo và gán giá trị cho biến

STT	Tên biến
1	\$name
2	\$Name
3	\$7up
4	\$_7up
5	\$năm_sinh
6	\$\$send
7	\$\$send@someone
8	\$send-to



Kiểu dữ liệu

- PHP hỗ trợ 4 kiểu dữ liệu đơn
 - Số nguyên
 - Số chấm phẩy động
 - Chuỗi
 - Logic
- Hai kiểu dữ liệu đa giá trị
 - Mảng
 - Đối tượng
- Kiểu dữ liệu tài nguyên (sử dụng khi tương tác dữ liệu)
- NULL : biến đặc biệt không có giá trị

Kiểu số

- Số nguyên từ -2^{31} đến $2^{31}-1$
 - Hệ thập phân: VD: `$a = 16;`
 - Hệ 16 (hexa): VD: `$a=0x10;`
 - Hệ 8 (bát phân): VD: `$a = 020;`
- Số thực (thập phân): từ $1.7E-308$ đến $1.7E+308$
 - Biểu diễn: `$a = 0.017`
 - Dạng khoa học: `$a = 17.0E-03`

Kiểu logic

- Có 2 trạng thái: `true` và `false`

Kiểu chuỗi

- Giới hạn bởi nháy đơn (') hoặc kép (")
- Chuỗi đặt trong nháy kép bị thay thế và xử lý ký tự thoát. Trong nháy đơn thì không.

- Ví dụ:

```
$a = "Hello";
```

```
$b = "$a world"; //tương đương $b="Hello world"
```

```
$c = '$a world'; //$c='$a world' (không thay đổi)
```

Kiểu chuỗi (tiếp)

- Để làm rõ các biến trong chuỗi, cần bao biến vào giữa cặp ngoặc nhọn { }

```
$a = "He";
```

```
$b = "$a11o"; //lỗi vì PHP hiểu là $a11o
```

```
$c = "{$a}11o"; //đúng ($c = "Hello")
```

- Biến mảng, biến biến khi viết trong chuỗi bắt buộc phải bao lại.

Kiểu chuỗi (tiếp)

- Kết hợp chuỗi (một tính năng quan trọng để tạo các Website động)

- Sử dụng dấu chấm (.)

- Ví dụ:

```
$first_name = "Nguyễn Văn";
```

```
$last_name = "A";
```

```
$name = $first_name." ".$last_name;
```

Kiểu chuỗi (tiếp)

- Ký tự thoát: \
 - Sử dụng để viết các ký tự đặc biệt trong chuỗi
 - VD:

Cần có chuỗi: Người ta nói "PHP rất tốt"

```
$a = "Người ta nói "PHP rất tốt""; //Sai
```

```
$a = "Người ta nói \"PHP rất tốt\""; //Đúng
```

- Một số ký tự phải sử dụng ký tự thoát: \$, \, "
 - Ngoài ra:
 - \n: Xuống dòng
 - \r: trở về đầu dòng
 - \t: dấu tab
 - ...

Kiểu chuỗi (tiếp)

- Kiểu tài liệu (heredoc):
 - Cho phép viết 1 chuỗi trên nhiều dòng.
 - Không cần sử dụng ký tự thoát:
 - Cách viết:

```
$biến = <<<Ký_hiệu  
    nội dung trên nhiều dòng  
Ký_hiệu;
```

- Chú ý: Ký_hiệu phải được viết ở ký tự đầu tiên của dòng
- Ví dụ:

```
$a = <<<EOD
```

```
    Đây là chuỗi nằm trên nhiều dòng sử dụng cú pháp kiểu tài  
    liệu 'heredoc'
```

```
EOD;
```

Kiểu mảng

- Cho phép chứa nhiều giá trị dữ liệu được đánh chỉ mục bằng số hay chuỗi.
- Để truy cập vào 1 phần tử, đặt chỉ số trong []
- Ví dụ:
`$a[0] = "Xin";`
`$a[1] = "Chào";`
- Có thể sử dụng chỉ số không liên tiếp
- Có thể thêm phần tử vào cuối mảng bằng cách bỏ chỉ số:
`$a[0] = "Xin";`
`$a[] = "Chào"; // $a[1]`
`$a[] = "Bạn"; // $a[2]`

Kiểu mảng (tiếp)

- Chỉ số chuỗi:

```
$a["ten"] = "Hồng";
```

```
$a["mau"] = "Đỏ";
```

- Mảng nhiều chiều

- Được coi là mảng của mảng

- Ví dụ:

```
$a[0][0] = "TT";
```

```
$a[0][1] = "Họ tên";
```

```
$a[1][0] = 1;
```

```
$a[1][1] = "Nguyen Van A";
```

Kiểu mảng (tiếp)

- Khởi tạo mảng bằng hàm `array()`

- Chỉ số mặc định:

```
$a=array("xin", "chào", "bạn");
```

```
/*tương đương với
```

```
$a[]="xin";
```

```
$a[]="chào";
```

```
$a[]="bạn"; */
```

Kiểu mảng (tiếp)

- Khởi tạo mảng bằng hàm `array()`

- Chỉ số mặc định với chỉ số đầu khác 0:

```
$a=array(5=>"xin", "chào", "bạn");
```

```
/*tương đương với
```

```
    $a[5]="xin";
```

```
    $a[]="chào";
```

```
    $a[]="bạn"; */
```

Kiểu mảng (tiếp)

- Khởi tạo mảng bằng hàm `array()`

- Chỉ số chuỗi:

```
$a=array("ten">"Hồng", "mau">"Đỏ");
```

/*Tương đương với:

```
$a["ten"] = "Hồng";
```

```
$a["mau"] = "Đỏ"; */
```


Kiểu mảng (tiếp)

- Khởi tạo mảng bằng hàm `array()`
 - Mảng nhiều chiều:

```
$a = array(  
    array("TT", "Họ tên"),  
    array(1, "Nguyễn Văn A")  
);
```

Ép kiểu và kiểm tra kiểu

- Sử dụng cú pháp tương tự C/C++

```
$x = "123abc";
```

```
// $x là chuỗi
```

```
$x = (int) "123abc";
```

```
// $x là số nguyên=123
```

- Kiểm tra kiểu:

```
gettype(), is_bool(),  
is_long(), ...
```

Ký hiệu	Ý nghĩa kiểu
(int), (integer)	Số nguyên
(real), (double), (float)	Số thập phân
(string)	Chuỗi
(array)	Mảng
(object)	Đối tượng
(bool), (boolean)	Logic
(unset)	NULL, tương tự như gọi unset()

Toán tử

- Kiểu số: +, -, *, /, %, **, ++, --
- Chuỗi: Toán tử chấm (.), .=
- Logic: AND (&&), OR (||), XOR, !
- Gán: =, +=, -=, *=, /=, %=
- So sánh: ==, ===, !=, <>, !==, <, >, <=, >=, <=>
- Mảng: +, ==, ===, !=, <>, !==
- Gán có điều kiện: ?:, ??

Các cấu trúc điều khiển

Rẽ nhánh - IF

```
if (điều kiện){  
    code block  
}
```

```
if (điều kiện){  
    code block  
}  
else {  
    code block;  
}
```

```
if (điều kiện 1) {  
    code block  
    khi điều kiện 1 đúng  
}elseif(điều kiện 2){  
    code block  
    khi điều kiện 2 đúng  
}else {  
    code block nếu tất cả  
    các điều kiện sai  
}
```

```
if ($quantity > 100) {  
    $Price = 1.00;  
}else if(quantity <50){  
    $Price = 1.20;  
}else {  
    $Price = 1.30;  
}
```

Rẽ nhánh - switch

```
switch(điều kiện) {  
    case GT1 :  
        code block  
        break;  
    case GT2 :  
        code block  
        break;  
    case GT3 :  
        code block  
        break;  
    default:  
        code block  
        break;  
}
```

```
switch ($quantity) {  
    case 50:  
        $unitprice = 1.2;  
        break;  
    default:  
        $unitprice = 1.2;  
        break;  
}
```

Lặp (while/ do-while)

```
while(condition) {  
    code block  
}
```

```
do {  
    code block  
} while(condition)
```

```
while ($i < 10) {  
    $text .= "The number is " . $i;  
    $i++;  
};
```

```
do {  
    $text .= "The number is " . $i;  
    $i++;  
}  
while ($i < 10);
```

Lặp xác định

```
for(statement 1; statement2; statement 3) {  
    code block  
}
```

```
for ($i = 0; $i < 5; $i++) {  
    $text .= "The number is " . $i . "<br>";  
}
```

```
foreach($array as $value){  
    code block  
}
```

```
foreach($array as $value){  
    echo $value;  
}
```

```
foreach($array as $key => $value) {  
    code block  
}
```

```
foreach($array as $key => $value) {  
    echo $key . ":" . $value;  
}
```


break và continue

- **break**: Kết thúc khối lệnh **for**, **while**, **do-while** hoặc **switch**
- **continue**: bỏ qua phần sau continue trong vòng lặp, chuyển sang vòng lặp tiếp.

Hàm tự tạo

- Khai báo hàm:

```
function tên_hàm([ts1, ts2,..., tsn]) {  
    //các lệnh trong thân hàm  
    [return return_value];  
}
```

- Để trả ra giá trị cho hàm ta sử dụng lệnh **return**, nếu không gán giá trị trả về thì giá trị NULL sẽ được trả về. Để hàm không trả về dữ liệu thì không viết lệnh **return**.
- Để thoát khỏi hàm:
 - **return**
 - **exit;**

Hàm tự tạo (tt)

- Gọi hàm:

`tên_hàm(gt1, gt2,... , gtn) ;`

- Chú ý:

- Nếu hàm không có tham số, khi khai báo hàm, gọi hàm vẫn phải giữ lại ()

```
<?php
    function nhanHaiso($so1, $so2) {
        $ketqua = $so1 * $so2;
        echo $ketqua;
    }
    nhanHaiso(2, 5);
?>
```

Gọi hàm động

- PHP cho phép lưu tên hàm như một chuỗi và gọi hàm thông qua chuỗi này.

```
<?php
    $func = 'nhanHaiso';

    nhanHaiso (2, 5) ;
    $func (2, 5) ;
?>
```

Hàm tự tạo – tham số mặc định

- Tham số hàm có thể được gán luôn cho nó một giá trị cụ thể để khi gọi hàm viết thiếu tham số này thì hàm được truyền giá trị mặc định.

```
<?php
function phepnhan ($num1, $num2=2) {
    $r = $num1 * $num2;
    return $r;
}
echo phepnhan (3) ;
echo phepnhan (3, 3) ;
?>
```

các tham số
mặc định ở phía
bên phải

Hàm tự tạo – tham số hàm với ký hiệu ...

- Hàm có thể được khai báo với số lượng tham số nhập vào tùy ý người dùng với ký hiệu dấu ba chấm “...” trước biến \$thamso:

```
function functionName ( ...$thamso) { }
```

- Biến \$thamso là biến mảng chứa các giá trị được truyền vào khi gọi hàm.

```
<?php
    function printNames (...$names) {
        foreach ($names as $name)
            echo $name . "<br>";
    }
    printNames ('Mr A', 'Mr B', 'Mr C');
?>
```

Hàm tự tạo – truyền tham số với ký hiệu ...

- Khi gọi hàm có thể sử dụng ký hiệu dấu “...” để sử dụng các phần tử của mảng truyền vào lần lượt làm các tham số của hàm:

```
<?php
    function add($a, $b) {
        return $a + $b;
    }
    echo add(...[1, 2])."\n";
    $a = [1, 2];
    echo add(...$a);
?>
```

Hàm tự tạo – parameters - arguments

- **Parameter:** tham số được đặt trong dấu ngoặc đơn () khi khai báo hàm, được sử dụng trong hàm như biến mà giá trị được cung cấp bởi lời gọi hàm.
- **Argument:** đối số là các giá trị truyền vào hàm qua lời gọi hàm.

```
<?php
    function addTwo($a, $b=2) {
        return $a + $b;
    }
    $a = 1;
    echo addTwo($a);
?>
```

Tham số
Parameter

Đối số
argument

Hàm tự tạo – độ dài tham số thay đổi

- Hàm trong PHP có thể làm việc với danh sách tham số có chiều dài thay đổi
- Xử lý qua đối số với các hàm:

Hàm	Mô tả
<code>func_get_args()</code>	Trả về một mảng chứa các tham số được truyền vào hàm
<code>func_num_args()</code>	Trả về số lượng tham số được truyền vào hàm.
<code>func_get_arg(\$i)</code>	Trả về tham số tại một chỉ mục cụ thể

Hàm tự tạo – độ dài tham số thay đổi

```
<?php
    function sum() {
        $acc = 0;
        foreach (func_get_args() as $n) {
            $acc += $n;
        }
        return $acc;
    }
    echo sum(1, 2, 3, 4);
?>
```

Truyền tham số theo tham trị

- Mặc định, tất cả các đối số của hàm được truyền theo giá trị (tham trị)
- **Truyền tham số theo tham trị:** một bản sao của đối số được gửi tới hàm. Khi hàm thay đổi tham số, nó chỉ thay đổi bản sao của đối số chứ không phải đối số ban đầu

```
<?php
    $a = 10;
    function binhphuong ($b)
    {
        $b *= $b;
        return $b;
    }
    echo binhphuong ();
    echo $a;
?>
```

Truyền tham số theo tham chiếu

- **Truyền tham số theo tham chiếu:** một tham chiếu đến các tham số ban đầu được gửi tới hàm. Khi hàm thay đổi tham số, hàm thực sự thay đổi các đối số ban đầu.
- Cú pháp: viết ký hiệu "&" trước tham số

```
<?php
    $a = 10;
    function binhphuong (&$b) {
        $b *= $b;
        return $b;
    }
    echo binhphuong ();
    echo $a;
?>
```

Phạm vi hoạt động của biến

- Biến cục bộ
- Biến toàn cục
- Biến siêu toàn cục
- Biến static
- Hằng

Biến cục bộ

- Biến được khai báo ***trong hàm hoặc là tham số của hàm***
- Khi ra khỏi hàm => biến cục bộ và giá trị của nó sẽ bị hủy bỏ
- *Lưu ý:* khi có cùng tên thì biến bên trong hàm và biến bên ngoài hàm là hai biến hoàn toàn khác nhau

```
<?php
$a = 5;
function test($b) {
    $a = $b;
    echo '$a trong hàm là: ' . $a;
}
echo test(10);
echo '$a ngoài hàm là: ' . $a;
?>
```

Biến toàn cục

- Có thể truy xuất bất cứ nơi nào trong trang
- Khi muốn sử dụng và cập nhật biến toàn cục trong hàm thì phải sử dụng từ khóa **global**

```
<?php
    $count = 10;
    function display_count() {
        global $count;
        $count++;
        echo $count;
    }
    count();
    echo $count;
?>
```

Biến siêu toàn cục

- Các biến siêu toàn cục được tạo sẵn và xác định trước và có thể được truy cập từ bất kỳ phạm vi nào.

- `$GLOBALS`
- `$_SERVER`
- `$_REQUEST`
- `$_POST`
- `$_GET`
- `$_FILES`
- `$_COOKIE`
- `$_SESSION`
- `$_ENV`

```
<?php
    $a = 5; $b = 6;
    function tinhtong() {
        $GLOBALS['c'] = $GLOBALS['a'] + $GLOBALS['b'];
    }
    tinhtong();
    echo $c;
?>
```


Biến tĩnh

- Biến được khai báo trong hàm với từ khóa ***static***
- Sẽ giữ nguyên giá trị trước đó khi hàm được gọi lại

```
<?php
function count_number ()
{
    STATIC $count = 0;
    $count++;
    echo $count;
}

count_number ();
count_number ();
count_number ();

?>
```

Khai báo hằng

- Hằng số là một tên (định danh) cho một giá trị đơn giản.
- Giá trị ứng với tên hằng sau khi định nghĩa là không đổi.
- Thông thường đặt tên hằng số bằng các ký tự IN HOA.
- Khai báo hằng:

```
define ("TÊN_HẰNG", <giá_trị>);  
const TÊN_HẰNG = <giá_trị>;
```

VD:

```
define ("FIRSTWEEKDAY", 'MONDAY') ;  
const MONDAY = "THỨ HAI";
```

Khai báo hằng

- Từ PHP 7 có thể định nghĩa hằng với một mảng các giá trị

```
const DAYOFWEEK = [ 'CN', 'T.HAI', 'T.BA', 'T.TU', 'T.NĂM', 'T.SÁU', 'T.BẢY', ];  
echo DAYOFWEEK[2];
```

- Một số hằng được định nghĩa sẵn:
 - `__LINE__`
 - `__FILE__`
 - `__DIR__`
 - `PHP_EOL`
 - `__FUNCTION__`
 - `__CLASS__`
 - `__METHOD__`
 - `__NAMESPACE__`

Một số hàm được cung cấp sẵn

Các hàm chèn file

- **include**("địa_chỉ_file");
- **require**("địa_chỉ_file");
- **include_once**("địa_chỉ_file");
- **require_once**("địa_chỉ_file");

Các hàm làm việc với biến

- Kiểm tra tồn tại: `isset()`
- Kiểm tra giá trị rỗng: `empty()`
- Kiểm tra kiểu dữ liệu của biến: `is_numeric()`, `is_int()`, `is_string()`, `is_array()`,...
- Xác định kiểu dữ liệu của biến: `gettype()`

Các hàm toán học

- Giá trị tuyệt đối: `abs()`
- Lũy thừa: `pow(số, lũy thừa)`
- Sinh ngẫu nhiên: `rand(min, max)`
- Làm tròn: `round(số, vị trí làm tròn)`
- Tính căn bậc hai: `sqrt()`

Các hàm xử lý chuỗi

- Định dạng chuỗi: ltrim(), rtrim(), trim(), strtoupper(), strtolower(), ucfirst(), ucwords(); nl2br()
- So sánh chuỗi: strlen(), strcmp(s1, s2), strcasecmp(s1,s2)
- Tìm kiếm và thay thế: strpos(str,s2), str_replace(s1,s2, str)
- Kết hợp và tách chuỗi: explode(k,s), implode(k,s), join()
- Mã hóa chuỗi: md5(str), htmlentities(str), html_entity_decode(), addslashes(str), stripslashes(str), htmlspecialchars(), htmlspecialchars_decode()

Các hàm xử lý mảng

- Kiểm tra mảng và giá trị: `is_array()`, `in_array(v,a)`
- Sắp xếp: `sort()`, `rsort()`, `array_reverse()`,
- Kích thước mảng: `array_keys()`, `count()`, `sizeof()`
- Cắt mảng: `array_shift()`, `array_unshift()`, `array_pop()`, `array_push()`,
- Duyệt mảng: `each(a)`, `list(k,v)`

Các hàm xử lý thời gian

- Kiểm tra ngày hợp lệ: `checkdate(m,d,y)`
- Định dạng ngày: `date(format,[time])`
- Lấy giá trị ngày, thời gian hiện tại: `getdate()`, `time()`
- Chuyển đổi ngày, thời gian: `strtotime('time_string')`,
`mktime(h,mi,s,m,d,y)`

Redirect

- Sử dụng hàm header

```
header( "Refresh:5; url=http://www.example.com/page2.php", true, 303) ;  
  
sleep(5) ;  
  
header("Location: page2.php", true, 303) ;
```

- Sử dụng javascript

```
<script> window.setTimeout(function() { window.location = 'page2.php'; }, 5000); </script>
```